A Missing Sign Symmetry Accounts for the Linear-Mode-Connectivity Barrier in Minimal-Width Sigmoid Networks

Mrigesh Lakhanpal
mrigeshlakhanpal4@gmail.com
ORCID: 0009-0007-9539-9091

October 2026 (preprint, v2)

This work is licensed under a Creative Commons Attribution 4.0 International license (CC BY 4.0).

Abstract

On two-dimensional classification tasks with separable labels and a convex-polygon positive region, permutation alignment fails to linearly connect independently trained minimal-width networks. Ainsworth et al. suggested that either their matching fails on thin models or "some form of invariance other than permutation symmetries" is at play; Benzing et al. noted that deciding this needs an exhaustive search. For irreducible one-hidden-layer sigmoid networks, the exact symmetry group (permutations and sign flips, $2^h h!$ elements at width h) is known and small enough to search exhaustively. In a first study (one task, width 3, 11 pairs), permutation matching left a median training-error barrier of 17.95 points, sign-aware matching 0.00, and the full search at most 0.11. A second, publicly pre-registered study used the original task and four new polygons (3–6 edges; 100–120 seeds each) at their measured minimal widths (3 or 4 units at a 2% error tolerance). On all five tasks, every pre-registered test of the main claim passed. In pairs needing a flip, the best permutation left a median of 9.95–21.65 points; the full search left an upper quartile of at most 0.13 (165 pairs); and the ordering held under training error, test error, raw loss and calibrated loss. By the pre-registered title rule, applied at the main 2% threshold, this gives the present title. Qualifications: the claim fails for one task at each sensitivity threshold (1%, 5%). In exploratory analysis, the 157 pairs that learned the same decision lines kept at most 0.21 points, and the 8 that did not kept 2.38–9.58. A pre-registered test of whether the weights predict barrier size failed for error barriers ($\rho = 0.007$), which saturate; an exploratory loss-based re-test gave $\rho = 0.86$ (calibrated: 0.65). ReLU networks have no sign flip and remain open.

Keywords: linear mode connectivity; permutation symmetry; sign-flip symmetry; sigmoid networks; loss landscape; model merging

1 Introduction

Two networks trained from different random initializations usually reach similar accuracy, yet the straight line between their weights typically passes through a region of high loss. Entezari et al. [1] conjectured that most of this barrier is an artifact of permutation symmetry: reorder the hidden units of one network appropriately, and SGD solutions become linearly mode connected [2]. Ainsworth et al. [3] gave strong evidence for wide networks, reaching zero-barrier connections between independently trained ResNets once they were wide enough. In narrow networks, however, permutation alignment does not remove the barrier, and, to my knowledge, the reason is not yet fully understood. Ainsworth et al. write that their result "suggests that either our permutation selection methods are failing to find satisfactory permutations on thinner models or

that some form of invariance other than permutation symmetries must be at play in the thin model regime" [3, Section 5.3], and they conjecture that permutation symmetries are "a necessary piece, though not a complete picture" [3, Section 7]. Benzing et al. observe the same failure in narrow fully connected networks and add that "a thorough falsification of the hypothesis in narrower networks would require an exhaustive (and therefore infeasible) search over the space of permutations" [4, Section 3.1]. Entezari et al. note that small networks lie outside the scope of their conjecture [1, Section 3.4]. The failure also appears beyond fully connected ReLU networks: on independently trained tanh CNNs with 16 channels per layer, Boufalis et al. find that permutation-only Git Re-Basin "still faces a significant performance barrier, with accuracy dropping sharply at the midpoint", which they relate to limited width and to the limits of weight matching [5, Section 5.2]. A missing sign symmetry is one possible contributor to that barrier, but their experiments do not isolate it, and I do not test CNNs. The thin models of Ainsworth et al. are ReLU networks; this paper answers the sigmoid analogue of their question, and the ReLU case remains open.

In general this question is difficult to settle, because the complete symmetry group of a network is either unknown or too large to search. One-hidden-layer sigmoid networks are an exception. For irreducible networks of this kind, the only exact symmetries are permutations of the hidden units and sign flips of individual units, which follow from $\sigma(-z) = 1 - \sigma(z)$ [6, 7]. The group has $2^h h!$ elements, only 48 at width $h = 3$, so the two explanations, *matching failed* and *another symmetry is involved*, can be told apart by trying every element.

I used this to study independently trained networks on two-dimensional tasks whose decision region is a convex polygon, so that every learned feature is a line that can be drawn. To my knowledge, this is the first study of linear mode connectivity in neural networks that searches the complete group of exact symmetries exhaustively, checks that the networks are irreducible so that the group is complete, and reports separately the barrier left by permutations alone and by permutations with sign flips; it covers one-hidden-layer sigmoid networks. Sign-aware alignment itself is not new [5, 8], and an analogous permutation/flip decomposition has been reported for soft tree ensembles [9] (Section 2). The contributions are:

1. **A missing symmetry in the matcher, not a failed search, explains the barrier at width 3** (Study 1, pre-registered; Section 4.2). On the coffee task (23 of 40 networks met the inclusion rule, 11 pairs), permutation matching leaves a median training-error barrier of 17.95 points, and exhaustive search over the 6 permutations still leaves 17.88. Matching with sign flips removes it (median 0.00), and an exhaustive search over all 48 symmetries leaves at most 0.11 points (post-hoc rule, dated before the search; Section 4.3).

2. **The result generalizes across tasks and minimal widths** (Study 2, publicly pre-registered; Section 5). On five tasks with measured minimal widths 3 and 4, every pre-registered test of the main claim passed under four barrier measures at the main 2% threshold, and it failed for one task at each sensitivity threshold. The minimal width depends on the error tolerance. In exploratory analysis, the residual stayed below ε in 328 of the 331 distinct pairs that learned the same lines.

3. **Negative results reported as such.** The pre-registered prediction that the weights alone predict the size of the barrier failed in error terms, for a reason the data show. In loss terms it held in an exploratory re-test (Section 5.4). The sensitivity runs contain two task–tolerance combinations where the claim fails.

The hypotheses and main questions were fixed before any network was trained, every decision taken after seeing results is labelled as such, and one command, including tests, rebuilds every result except the optional exhaustive search in about 87 minutes on a laptop CPU, reproducing the main result files byte for byte on the same machine (see the Reproducibility statement).

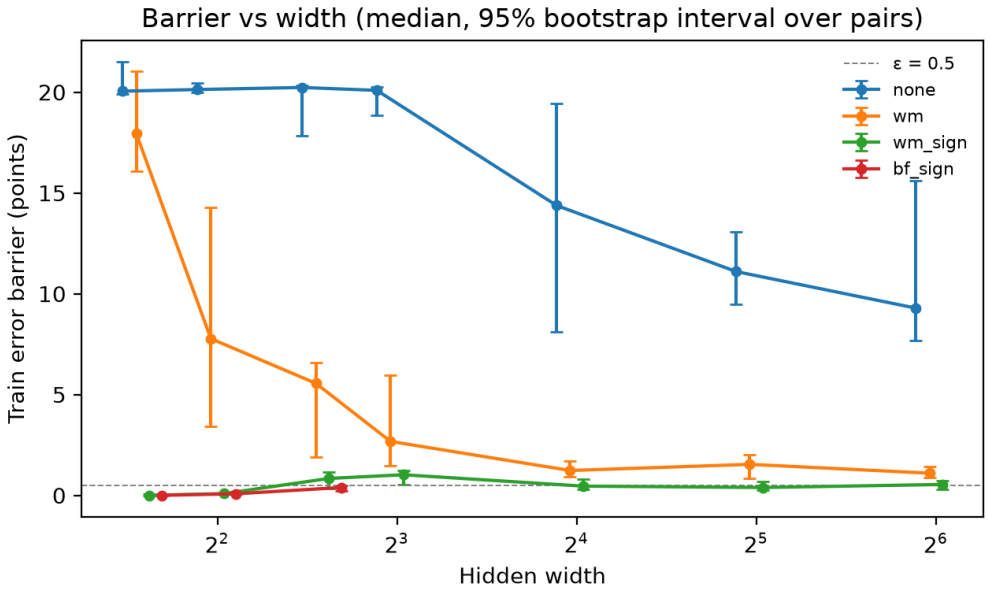


Figure 1: Study 1 (coffee task). Training-error barrier against hidden width: median over independent pairs, with 95% bootstrap intervals (10,000 resamples of pairs). none: no alignment. wm: weight matching over permutations. wm_sign: weight matching over permutations and sign flips. bf_sign: brute force over all permutations and flips (widths 3–6), chosen by midpoint loss. Dashed line: $\varepsilon = 0.5$ points. Width 3: 11 pairs; widths 4–64: 20 pairs; no width-2 network met the inclusion rule.

2 Background and related work

Barriers. For networks with parameters θ_A, θ_B and a loss or error L, the barrier along the straight line is [1, Eq. 1]

$$B(\theta_A, \theta_B) = \sup_{\alpha \in [0,1]} \Big[L\big((1-\alpha)\theta_A + \alpha\theta_B\big) - \big((1-\alpha)L(\theta_A) + \alpha L(\theta_B)\big) \Big], \tag{1}$$

so that $\alpha = 0$ is network A. Entezari et al. report it at $\alpha = 1/2$ as a surrogate for the supremum [1, footnote 3]. I evaluate it on a grid of 25 values of α and use the training *error* as L for all verdicts (Section 3).

Symmetries of one-hidden-layer sigmoid networks. Write the network as

$$f(x) = \sum_{i=1}^{h} s_i \, \sigma(w_i^\top x + b_i) + c, \qquad \sigma(z) = \frac{1}{1+e^{-z}}, \tag{2}$$

with input weights w_i, biases b_i, output weights s_i and output bias c. Permuting the units leaves f unchanged [10]. Because $\sigma(-z) = 1 - \sigma(z)$, so does a *sign flip* of unit i:

$$(w_i, b_i, s_i, c) \;\mapsto\; (-w_i, -b_i, -s_i, c + s_i), \qquad \text{since} \qquad -s_i\sigma(-z) + c + s_i = s_i\sigma(z) + c. \tag{3}$$

This group of permutations and sign flips was described for tanh networks by Chen et al. [11]. Sussmann proved that for irreducible one-hidden-layer tanh networks these are the only symmetries [6]; Vlačić and Bölcskei restate the lemma behind it, on the symmetries of tanh itself, as their Lemma 2 and study identifiability for broader classes of activations [7]. For sigmoid and tanh, Martinelli et al. [12] show that a neuron's input weights are identifiable from the network's function only up to sign. Their Section 4 groups both as

"odd (+ constant)" activations, for which a unit can be replaced by one with flipped signs, the constant being absorbed by the next layer's bias (Appendix A.2.2; their table in Appendix A.2.3 lists tanh as odd and the sigmoid as odd plus constant); Theiss et al. refine this classification and call such activations constant-odd [13, Appendix B.3]. Since $\sigma(z) = \frac{1}{2}\big(1 + \tanh(z/2)\big)$, a sigmoid network with an output bias is a tanh network with rescaled weights, so the same holds for Eq. (2). A network is irreducible when no output weight is zero, no unit has zero input weights, and no two units have equal or opposite input parameters (w_i, b_i) [6] (the last condition is Lemma 1 of [7]); in practice I also flag units that are nearly constant on the data. Two limits matter here. First, uniqueness holds for the function on all of $\mathbb{R}^2$; on a finite data set, the set of equivalent networks can only be larger [7, Section I-A]. Bürgin et al. [14] study exactly such approximate, data-dependent symmetries and their effect on linear mode connectivity. Second, if a network is *nearly* reducible, for example because two units almost coincide, it has continuous near-symmetries that no finite search covers.

Related work. Entezari et al. ran a brute-force permutation search on small ReLU networks trained on MNIST [1, Appendix E.3], and Ainsworth et al. exhaustively searched the permutations of a hand-built two-hidden-layer ReLU network on a two-dimensional task to show that some solutions admit no linearly connecting permutation [3, Appendix A.6]. ReLU networks have no sign-flip symmetry, so flips do not arise in either setting. Martinelli et al. already use the sign flip to align independently trained networks: before clustering the neurons of several student networks, they "arbitrarily align all input weight vectors to have a positive n^{th} element" [12, Section 5]. Pittorino et al. already removed sign-reversal symmetries together with permutations when aligning independently trained networks with sign activations: they match absolute cosine similarities, apply the permutation, and then flip each unit if that increases the similarity, and they find that removing symmetries lowers the barriers [8, Section 3.1.2]; their networks were wide and binarized, the matching heuristic, and the permutation and sign contributions were not separated. Wiese et al. likewise remove tanh sign flips and then permutations from Markov chain Monte Carlo samples of Bayesian networks, one of which has a single hidden layer of three tanh units [15, Section 5.3 and Appendix B]. Boufalis et al. [5, Appendix A, Proposition 1] extended weight matching to sign flips for sine and tanh networks, essentially the `wm_sign` mode used here. They used this sign-aware matcher, exact for one hidden layer and an approximation in deeper networks, only on networks constructed to be equivalent: transformed copies of one sine network with small perturbations (their Section 5.1); in their Appendix A they note that it can flip signs unnecessarily. On independently trained tanh CNNs, their Git Re-Basin baseline was permutation-only. Their sign-aware method there was a learned ScaleGMN autoencoder whose decoded networks approximate the originals (their Table 2), so its aligned endpoints are not exact symmetry copies. In a different model class, Kanoh and Sugiyama showed that, for soft decision-tree ensembles, linear mode connectivity requires accounting for tree-specific invariances beyond permutations: a "subtree flip invariance", which rests on the same identity $\sigma(-c) = 1 - \sigma(c)$, and a splitting-order invariance; they "performed an exhaustive search to explore all patterns" of these invariances [9, Section 3.2]. Building on these works, I apply exact sign-aware matching to independently trained networks and certify it with an exhaustive search over the complete group. Şimşek et al. [16] assumed that at minimal width the global minima are isolated permutation copies, and showed that one extra neuron connects them through straight segments (their Theorem 3.1 and Corollary 4.1); for sigmoid, they remark, about their related Theorem 4.2, that "the analysis must include additional invariances" (Remark 4.1). Section B builds on their construction by adding a sign-flip move. Zhan et al. [17] analyze a two-layer ReLU teacher–student model with fixed output weights, no biases and a population loss; from their characterization of the global minima (Lemma 2, from [18]) they note that when student and teacher widths are equal, any two minima coincide up to permutation, so the permuted barrier is zero (their Figure 4 caption). My width-3 result can be seen as an empirical sigmoid analogue, in which the coincidence holds only up to permutations *and* sign flips. Zhao et al. [19] relate the connected components of the set of minima to the

topology of the symmetry group, build constant-loss curves from continuous symmetries (their Section 6.1), and note that characterizing the minima of nonlinear networks "requires identifying the complete set of symmetries" (Section 7). For irreducible one-hidden-layer sigmoid networks that complete set of exact symmetries is known, and this study searches all of it. Ito et al. [20] show that very wide image networks become linearly connected without any permutation after temperature calibration; I return to their work in Section 6.

3 Setup and pre-registration

Task and data. The task is the "coffee roasting" problem from the Machine Learning Specialization [21]: two features (temperature 150–285 °C, duration 11.5–15.5 min), and a roast is good inside a triangle bounded by $t > 175$, $d > 12$ and $d \leq -\frac{3}{85}t + 21$. The data generator is a re-implementation written for this study; it uses the same labeling rule as the course's `load_coffee_data`, checked against a publicly available, unofficial copy of the lab file, since the official copy was not accessible to me; no course files are redistributed with this work. Unlike the lab, which uses 200 points, I draw 2,000 fresh training points and 2,000 test points (seeds 0 and 1); 20.95% of the training points are good roasts. Features are standardized with the training statistics. The good region has exactly three edges, so three hidden units are the natural minimum.

Networks and training. Networks follow Eq. (2) with a sigmoid output and binary cross-entropy. I trained widths 2, 3, 4, 6, 8, 16, 32 and 64, with 40 seeds each, using Glorot-uniform weights and zero biases [22] and full-batch Adam [23] (learning rate 0.01, 3,000 steps, no weight decay). Because training is full batch, the initialization is the only source of randomness. Before the main run I checked the learning rate against a rule fixed in advance (loss always finite; no rebound of more than 10% after step 200; final loss within 5% of the lowest); all 10 check runs (5 seeds at widths 3 and 64) passed.

Inclusion and pairing. A network is included if its training error is at most 2%. Included networks are paired disjointly in seed order, so pairs are independent samples. Inclusion was 0 of 40 at width 2, 23 of 40 at width 3 (11 pairs) and 40 of 40 at every larger width (20 pairs). Throughout, "minimal width" is meant operationally: width 3 is the smallest tested width at which networks met the inclusion rule under this training protocol. I do not prove that no width-2 sigmoid network can classify the task; three triangle edges motivate the choice but are not a lower-bound proof.

Barrier. I evaluate Eq. (1) at 25 evenly spaced values of α. The *training-error barrier*, in percentage points (1 point = 20 of 2,000 training examples), decides every verdict, with $\varepsilon = 0.5$ points (10 training examples) as the threshold for "removed". The test-error barrier is reported alongside. In Study 1, loss barriers are reported only after output-scale calibration (Section 6); Study 2 reports raw and calibrated loss barriers side by side.

Alignment modes. Before interpolating, network B is aligned to network A in one of five ways:

- none: no alignment.
- wm: weight matching [3]. For one hidden layer this is a single linear assignment problem: I represent each unit by $u_i = (w_i, b_i, s_i) \in \mathbb{R}^4$ and choose the permutation maximizing $\sum_i u_i^{A\top} u_{\pi(i)}^B$ (Hungarian algorithm [24]).
- wm_sign: the same with sign flips allowed, solved by maximizing $\sum_i |u_i^{A\top} u_{\pi(i)}^B|$ and flipping unit $\pi(i)$ by Eq. (3) when its inner product is negative, as in [8, Section 3.1.2] and [5, Proposition 1]. The output-bias shift of a flip is not part of the matching objective.

Table 1: Pre-registered hypotheses (H) and questions (Q). A line "matches" if its angle differs by at most 10° and its offset by at most 0.25 standardized units.

	Prediction or question	Falsified if
H1	More than half of converged width-3 pairs learn the same three lines	Half or fewer of the pairs match on every line
H1b	Seeds with higher training error have lines further from the other seeds'	Spearman $\rho \leq 0$ over all 40 width-3 seeds
H2	The wm-aligned blend at width 3 is slightly worse, about 1.8 points	The median falls outside 0.5–3 points
H3	The aligned (wm) barrier shrinks with width	Width 64's median is not below width 3's (a rise-then-fall counts as a partial miss)
Q1	Do independently trained width-3 networks learn the same lines?	—
Q2	How does the barrier change with width?	—
Q3	Does weight matching find the barrier-minimizing alignment?	—
Q4	Do sign flips lower the barrier beyond permutations?	—

- bf_perm and bf_sign: brute force over all permutations (widths up to 8) or all permutations and flips (widths up to 6), choosing the alignment with the lowest loss at the midpoint, the surrogate of [1].

Two further modes, exact_perm and exact_sign, choose by the full 25-point error barrier instead; these belong to the post-hoc analysis of Section 4.3.

Pre-registration. The hypotheses and questions in Table 1 were fixed in my design sessions of 4–5 October 2026 and recorded in my research notebook before any network was trained. The numerical thresholds (inclusion rule, line-matching tolerances, ε, the H2 band, and the H1b distance and test) were dated 7–8 October, before the main run on 9 October. Three things were *not* dated in advance: the pass fractions named in the falsifiers of Q1, Q3 and Q4, and which aligned mode Q2 should use. None of the answers below depends on them except Q3, which I therefore report as counts only, and Q2, which I therefore report for both aligned modes. This was an informal pre-registration: the record is a dated private research notebook, not a public registry such as OSF. The project was not under version control at the time, so there is no pre-registration commit; the dated notebook is the record, and it is available from me on request. Everything decided after seeing results is labelled *post-hoc* below. The Study 1 rules are now public in the repository (`preregistration/v1_rules_2026-10.md`); their dates come from the notebook and are not independently timestamped. The Study 2 rules were committed to the public repository before any Study 2 network was trained (Section 5).

4 Results

All values are medians over pairs with 95% percentile bootstrap intervals [25] (10,000 resamples of pairs); paired differences are medians of per-pair differences. Appendix A lists every value.

4.1 Width-3 networks learn the same three lines (H1 supported, H1b supported)

All 11 of 11 width-3 pairs matched on every line (largest angle difference 2.0°, largest offset 0.12 standardized units; Figure 2), so H1 is supported and the answer to Q1 is yes. For comparison, 16 of 20 width-4 pairs

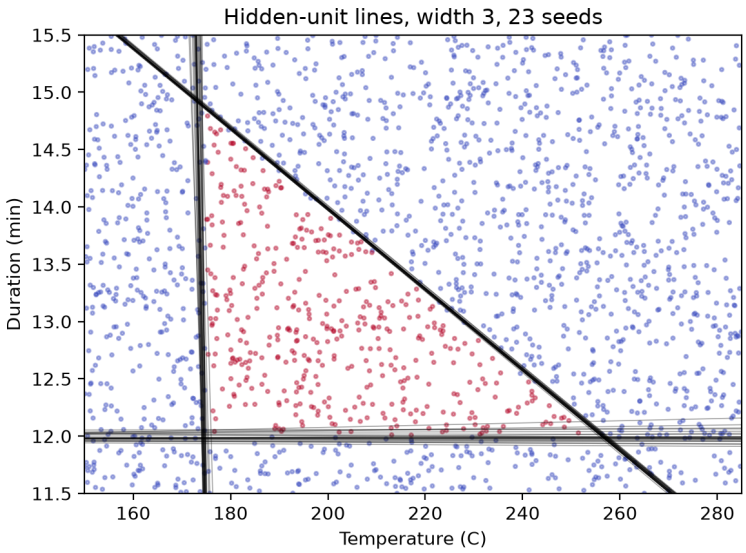


Figure 2: The three hidden-unit lines of all 23 included width-3 networks, drawn in original units over the training data (red: good roasts). Independently trained networks learn the same three edges of the triangle.

matched. The claim covers networks that finished training: at a looser 5% inclusion rule, 10 of 12 width-3 pairs matched, and the two that did not match differed by 39.5° and 23.9°.

For H1b, the Spearman rank correlation [26] between a seed's training error and its median distance to the other seeds' lines (in *threshold units*: for the worst-matched line, the larger of its angle difference divided by 10° and its offset difference divided by 0.25, so that 1 is the edge of the H1 tolerance) is $\rho = 0.800$ [0.625, 0.886] over all 40 width-3 seeds, so H1b is supported. The support comes from two separate groups rather than a smooth trend: 23 seeds that finished training (error 0.65–1.3%, distance 0.20–0.52 threshold units) and 17 that did not (error 3.65–16.85%, distance 2.4–10.8). Within each group the correlation is weak ($\rho = 0.17$ and 0.42).

4.2 Permutations leave the barrier; sign flips remove it (H2 not supported, Q4 yes)

Although the width-3 networks compute the same three lines, the median wm barrier is **17.95 [16.07, 21.05] points**, far outside the 0.5–3 band I predicted, so H2 is not supported. Without alignment the median is 20.05 points. Reordering hidden units alone does not align these networks.

Allowing sign flips changes the picture (Figures 1 and 3). The pre-registered Q4 measure compares the best permutation (bf_perm) with the best permutation and flip pattern (bf_sign): flips lowered the training-error barrier by more than ε in **10 of 11** pairs at width 3, 16 of 20 at width 4 and 16 of 20 at width 6, with median drops of 17.79, 7.72 and 3.37 points (paired interval at width 3: -17.79 [-20.18, -15.14]). The best sign-aware alignment used at least one flip in 10 of 11 width-3 pairs. Descriptively (post-hoc), 0, 1, 2 and 3 units were flipped in 1, 5, 4 and 1 pairs; each network appears to settle on either orientation of each line arbitrarily. This says nothing about the mechanism, since symmetric initialization alone would produce it. The wm_sign barrier at width 3 is 0.00 [0.00, 0.05]. Beyond the Q4 measure, the per-pair difference wm_sign $-$ wm has its whole interval below zero at every width, from -17.84 [-21.05, -16.07] at width 3 to -0.61 [-0.93, -0.38] at width 64.

The mechanism is visible in Eq. (3). Two networks that differ by a flip of unit i compute the same function, but halfway along the straight line between them that unit's input weights, bias and output weight all blend to zero. Its contribution to the logit then changes by $-s_i(\sigma(z) - \frac{1}{2})$, which is up to $|s_i|/2$ in size. On

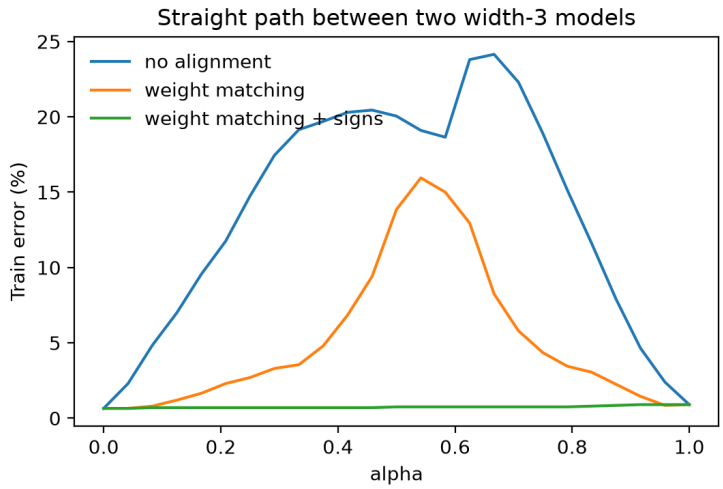


Figure 3: Training error along the straight path between the first pair of width-3 networks ($\alpha = 0$: network A), with no alignment, with weight matching, and with weight matching plus sign flips.

Table 2: Post-hoc complete-group search (exact_sign): training-error barrier left after the best of all $2^h h!$ permutation-and-flip combinations, chosen by the full barrier.

Width	Pairs	Residual: median / upper quartile / max	Pairs above ε	Pairs with a nearly reducible network
3	11	0.00 / 0.04 / 0.11	0	0 of 11
4	20	0.08 / 0.11 / 1.44	1	20 of 20
6	20	0.33 / 0.52 / 1.78	5	20 of 20

one trained width-3 network (seed 1, unit 2, $s = -11.14$), the straight blend to its own flipped copy changes the logit by up to 5.57 and reaches 20.95% training error (Figure 4). Permutation matching cannot undo such a flip; it can only choose which unit is paired with which.

4.3 Post-hoc: an exhaustive search over the complete group

After the pre-registered questions were answered, I asked how large a barrier remains after searching *every* permutation and sign flip and choosing by the full 25-point error barrier rather than the midpoint. The search is exhaustive over the transformations; the barrier itself is evaluated on this grid of 25 interpolation weights, so it is a lower bound on the maximum over the continuous segment. The rules for this analysis, including the irreducibility checks and the threshold for "removed" (upper quartile at most ε), were dated on 9 October, before the search ran. A network was flagged *nearly reducible* if its smallest output weight was below 5% of its largest, if two of its own units matched under the same line tolerance used for H1, or if a unit's activation varied by less than 0.01 (standard deviation) over the training data.

At width 3, the residual is at most **0.11 points** in all 11 pairs (Table 2). Every included width-3 network is irreducible: the closest pair of units within a network is at a median of 4.2 threshold units, where 1 would mean the same line. The search therefore covers every exact symmetry these networks have, and exact symmetries account for essentially the whole barrier. This does not show that the two networks of a pair compute the same function: identifiability fixes the symmetries available to each network, and the small residual shows only that the best of them brings the pair close. The result holds at 1% and 5% inclusion (maximum residual 0.09 and 0.47 points; no included network nearly reducible).

At widths 4, 6 and 8, nearly every network has two units on almost the same line, with a spare unit

Table 3: Q3: weight matching against brute force (pairs where weight matching is the same / worse / better, within ε). Left: pre-registered comparison with midpoint-chosen brute force. Right: post-hoc comparison with the exact search. —: not computed; the signed search was not run at width 8 (10.3 million candidates per pair; see Section 4.3).

	Pre-registered (midpoint)		Post-hoc (exact)	
Width	wm vs. bf_perm	wm_sign vs. bf_sign	wm vs. exact_perm	wm_sign vs. exact_sign
3	8 / 2 / 1	11 / 0 / 0	7 / 4 / 0	11 / 0 / 0
4	11 / 4 / 5	19 / 1 / 0	15 / 5 / 0	19 / 1 / 0
6	14 / 6 / 0	11 / 9 / 0	12 / 8 / 0	11 / 9 / 0
8	9 / 10 / 1	—	8 / 12 / 0	—

copying one of the three lines the task needs (37 of 40 networks at width 4, 38 of 40 flagged by any rule, and all networks at widths 6 and 8). Such networks may have near-symmetries outside the finite group, exact or holding only on the data, so their residuals are upper bounds only; I stated this risk before the search. Width 6 falls short of the "removed" threshold by 0.02 (upper quartile 0.517). I decided after seeing the width-6 result not to run the signed search at width 8 (10.3 million candidates per pair), since its networks are all nearly reducible.

For the question raised by Ainsworth et al. [3, Section 5.3], the answer *for these sigmoid networks on this task* is *another symmetry*: permutation matching leaves a large barrier because a sign flip is not a permutation, and once flips are included, both weight matching and the exhaustive search remove the barrier at width 3. A sign flip is an affine relation between activations, since the flipped unit outputs $1 - \sigma(z)$; this fits their own hypothesis that "linear, possibly non-permutation, relationships connect the layer-wise activations between models trained by SGD" [3, Section 7]. ReLU networks have no sign-flip symmetry, so this explanation does not carry over to the ReLU networks of their experiments, where the question remains open.

4.4 Width dependence (H3's endpoint prediction supported, Q2)

The median wm barrier falls from 17.95 points at width 3 to **1.10 [0.86, 1.42]** at width 64, so H3's endpoint prediction is supported. The observed trend is broadly decreasing but not monotonic: the median rises by 0.31 points between width 16 (1.23) and width 32 (1.54), below ε, with overlapping intervals ([0.91, 1.68] and [0.83, 2.03]). The pre-registered falsifier did not say how to treat a small rise after a long fall, so I report the non-monotonicity rather than classify it.

For Q2: without alignment the median barrier stays near 20 points up to width 8 and then falls to 9.29 [7.67, 15.60] at width 64. With wm it shrinks, but its upper quartile stays above ε at every width (1.61–20.31). With wm_sign the barrier is removed at widths 3 and 4 (upper quartiles 0.04 and 0.11), but not from width 6 to 64 (upper quartiles 0.77–1.30, medians 0.39–1.02). Reporting both aligned modes was a post-hoc decision, because Q2 did not name one. The test-set barrier gives the same answers at every width.

4.5 Does weight matching find the best alignment? (Q3, counts only)

Table 3 compares weight matching with brute force, counting a pair as "same" when the two barriers differ by at most ε. Under the pre-registered comparison, weight matching "beat" brute force in 7 pairs, by up to 20 points. This is possible only because brute force chose by the midpoint. Entezari et al. found the midpoint an accurate surrogate in their settings [1, footnote 3], but in these small networks the error peak sometimes lies elsewhere: in all 7 pairs it lay more than 0.05 from $\alpha = 0.5$ ($\alpha = 0.375$ to 0.708). The post-hoc exact search removes this artifact. With sign flips, weight matching finds the best alignment at widths 3 and 4 and drifts from it as width grows. With flips, the midpoint and full-barrier rules never differed by more than ε.

5 Study 2: five tasks and their minimal widths (pre-registered)

Study 1 (Sections 3–4) rests on one task, one width and 11 pairs, with the decisive search chosen after the main run. Study 2 asks whether its result holds on new tasks, at their own minimal widths, with more pairs, under rules fixed in public beforehand.

Pre-registration and firewall. The Study 2 rules (`preregistration/v2_rules.md`) were committed to the public repository (commit `349aa3a`, 19 October 2026) before any Study 2 network was trained. Training (Stage 1) used training error only; its outputs were then frozen with a SHA-256 fingerprint of every model file (commit `4c07b33`), and the analysis code refuses to run on changed files. Clarifications and deviations are logged with dates in `preregistration/DEVIATIONS.md` (D1–D9). D7 was written after the training counts and before any barrier, and D8 and D9 after all results; all three are marked below. The rules also fix the title in advance: the present title if H4a holds on every analysable task (the coffee task and at least three polygons), "… in Small Sigmoid Networks" if it holds on some, and "… in Width-3 Sigmoid Networks" if only on the coffee task.

Tasks. T0 is the coffee task of Study 1, with 80 new seeds added at width 3 (120 in total). T3–T6 are convex polygons with $k = 3, 4, 5, 6$ edges, drawn once by a fixed rule: irregular, off-centre, area 21% of the square $[-1, 1]^2$ to match the coffee task's positive rate, every interior angle at most 165° and every edge at least 0.25 long. Inputs are uniform on the square (2,000 training and 2,000 test points); labels are exact, so the data are separable. Model, initialization, optimizer, inclusion rule (2%), pairing and barrier are exactly as in Study 1. Each polygon task trains 100 seeds at widths $k-2$ to $k+2$ (at least 2). The original 40 coffee seeds were retrained to record their error 300 steps before the end. Their first 11 pairs reproduce Study 1's residuals exactly (0.03, 0, 0, 0.05, 0, 0.08, 0.11, 0.01, 0, 0, 0 points). The coffee task's 36 pairs therefore include Study 1's 11, so T0 is a re-test on an enlarged sample, not an independent replication; on the 24 pairs made only of new seeds (40–119) the residual is median 0.03, upper quartile 0.05, maximum 0.17 points. The four polygon tasks are new.

Minimal width is measured, not assumed. h^* is the smallest width at which at least 10 networks met the inclusion rule. At 2% it is 3 for both triangles and 4 for the quadrilateral, the pentagon *and* the hexagon. At 1% it equals k for every polygon. The width grid starts at $k-2$, so for the hexagon at 2% (and the hexagon and pentagon at 5%) h^* is the smallest width trained and is an upper bound, not a measurement. Four sigmoid units can draw four lines that cut off the small corners of a pentagon or hexagon and still misclassify fewer than 40 of 2,000 points. "Minimal width" is therefore tied to an error tolerance, and I report both.

Analysis. At h^*, every disjoint pair of included networks was aligned by none, wm, wm_sign, and by exhaustive search over all permutations (exact_perm) and all permutations and flips (exact_sign), each chosen by its 25-point training-error barrier (the Study 1 rule). Each alignment was scored by four measures: training-error barrier (deciding), test-error barrier, raw loss barrier, and loss barrier calibrated over 81 output scales from 0.01 to 16. The pre-registered hypotheses are:

- **H1**: all pairs learn the same lines.
- **H4a**: the exact_sign residual has an upper quartile $\leq \varepsilon$.
- **H4b**: in pairs that need a flip, exact_perm leaves a median above 2 points.
- **H4c**: wm_sign is within ε of exact_sign in at least 80% of pairs.
- **H5**: exact_perm $>$ exact_sign in at least 90% of flip pairs under each of the four measures.

Table 4: v2 main analysis (2% inclusion), at the measured minimal width h^*. Residual: exact_sign training-error barrier, median / upper quartile / maximum, in points. Perm-only: median exact_perm barrier in pairs that need a flip. H4c: share of pairs where wm_sign is within ε of exact_sign. H5: smallest share, over the four measures, of flip pairs where exact_perm > exact_sign. Every network was irreducible (no pair flagged). Generated from `results_v2/`.

Task	h^*	Pairs	Same lines	Residual	Perm-only	Flip pairs	H4c	H5
Coffee (triangle)	3	36	36/36	0.02 / 0.05 / 0.17	19.98	32	100%	100%
Triangle	3	47	47/47	0.03 / 0.06 / 0.13	21.65	36	100%	100%
Quadrilateral	4	32	30/32	0.04 / 0.07 / 9.58	17.53	26	97%	96%
Pentagon	4	21	21/21	0.05 / 0.07 / 0.15	12.38	14	100%	100%
Hexagon	4	29	23/29	0.06 / 0.13 / 4.81	9.95	21	97%	100%

- **H7**: the weights predict the size of the barrier (Section 5.4).

Each threshold's justification is in the rules file. Inclusion counts per width, flip counts and results above h^* are in Appendix D.

5.1 Pre-registered verdicts

H4a, H4b, H4c and H5 hold for all five tasks (Table 4). Allowing sign flips turns a median barrier of 9.95–21.65 points into an upper-quartile residual of at most 0.13 points. In 129 of 165 pairs the best alignment needed at least one flip. The ordering holds under every measure, so it is not an artifact of the error metric saturating. H1 fails for the quadrilateral (2 of 32 pairs) and the hexagon (6 of 29); the next subsection shows what happens in those pairs. For the hexagon this meets item 3 of the pre-registered list of outcomes that would count against the account ($h^* < k$ with H1 failing): four units suffice, and 6 of 29 pairs learn different lines. Item 4 did not occur (14 of 187 excluded networks were classed as wrong minima). Under the pre-registered title rule, these verdicts give the present title. After exact alignment, the networks of a same-line pair remain genuinely different parameter vectors (relative distance 0.04–0.66; median over all pairs 0.23), yet the barrier closes: the symmetry, not proximity, does the work.

Sensitivity (pre-registered at 1% and 5%; h^* re-derived by D7). Table 5. At 1%, $h^* = k$ for every polygon, and the claim holds for every task except the pentagon at width 5. At 5%, the claim holds for every task except the hexagon at width 4. Under the title rule, each sensitivity run alone would give "Small Sigmoid Networks". The title follows the rule at the main 2% threshold, as pre-registered, and these two failures are part of the result.

5.2 Exploratory: learning the same lines suffices

This split was decided after seeing the results (D9). Splitting the exact_sign residual by whether a pair passed the same-line test (Table 6):

- At 2%, all 157 same-line pairs end at or below 0.21 points, across all five tasks.
- The 8 pairs that learned different lines keep 2.38–9.58 points.
- Both sensitivity failures coincide with many different-line pairs: 7 of 18 pentagon pairs at 1% and 30 of 48 hexagon pairs at 5%.

The split runs mainly one way. Over the three thresholds, 328 of 331 distinct same-line pairs ended below ε (maximum 0.74 points). A pair that fails the line test, however, can still be nearly connected when its lines

Table 5: Inclusion sensitivity. h^* is re-derived at each threshold from training error only (D7). Residual: upper quartile of the exact_sign training-error barrier, in points. Pairs and same-line counts exclude pairs with a nearly reducible network (two, for the hexagon at 5%).

Inclusion	Task	h^*	Pairs	Same lines	Residual (UQ)	H4a
1%	Coffee (triangle)	3	28	28/28	0.04	yes
1%	Triangle	3	46	46/46	0.06	yes
1%	Quadrilateral	4	28	28/28	0.07	yes
1%	Pentagon	5	18	11/18	3.26	**no**
1%	Hexagon	6	30	22/30	0.09	yes
2%	Coffee (triangle)	3	36	36/36	0.05	yes
2%	Triangle	3	47	47/47	0.06	yes
2%	Quadrilateral	4	32	30/32	0.07	yes
2%	Pentagon	4	21	21/21	0.07	yes
2%	Hexagon	4	29	23/29	0.13	yes
5%	Coffee (triangle)	3	38	34/38	0.05	yes
5%	Triangle	3	47	47/47	0.06	yes
5%	Quadrilateral	3	48	44/48	0.22	yes
5%	Pentagon	3	46	46/46	0.18	yes
5%	Hexagon	4	48	18/48	2.58	**no**

Table 6: Exploratory (D9): exact_sign residual (points) split by the same-line test, pooled over the five tasks. A pair is counted once per threshold, so a pair can appear in more than one row; over the three thresholds there are 331 distinct same-line pairs (3 above ε) and 58 distinct different-line pairs (23 at or below ε).

	Same lines				Different lines		
Inclusion	Pairs	Median	Max	$> \varepsilon$	Pairs	Median	Range
2%	157	0.03	0.21	0	8	3.74	2.38–9.58
1%	135	0.03	0.20	0	15	0.17	0.00–4.27
5%	189	0.06	0.74	3	38	1.06	0.02–6.06

differ only slightly beyond the tolerance: 23 of 58 distinct different-line pairs ended below ε (median 0.17 points at 1%). The split therefore also depends on the line tolerance (10°, 0.25), which defines H1.

This links the empirical result to identifiability: if two irreducible networks compute the same function, Sussmann's theorem [6] leaves only permutations and flips, and the question becomes whether trained networks reach the same function. In these data, pairs that learned the same lines were almost always closed by the exact group; pairs whose lines clearly differed kept a barrier (2.38–9.58 points at 2%).

5.3 Networks that failed the inclusion rule

At h^*, each network above 2% training error was classified by a pre-registered rule, first match wins:

- **degenerate**: nearly reducible;
- **not converged**: its error still changed in the last 300 steps;
- **wrong minimum**: some edge of the polygon has no matching unit line.

Under this order, 148 of 187 were classed not converged, 14 wrong minima and 25 degenerate (Table 7). The classification separates less than it seems. 147 of the 148 networks classed not converged also miss a polygon edge, and in 20 of them the error rose rather than fell in the last 300 steps. At $h^* = 4$ for the pentagon and

Table 7: Classes of networks that failed the 2% inclusion rule at h^* (pre-registered classification).

Task	Failed inclusion	Not converged	Wrong minimum	Degenerate	Other
Coffee (triangle)	48	35	1	12	0
Triangle	6	4	2	0	0
Quadrilateral	35	26	2	7	0
Pentagon	57	48	5	4	0
Hexagon	41	35	4	2	0

hexagon, every network misses an edge by construction. I therefore do not claim that separate basins are rare. The claim is stated for networks that reach the inclusion threshold. Including every run regardless of inclusion (exploratory) raises the exact_sign residual to a median of 0.03–1.22 points by task, as expected for unfinished networks.

5.4 H7: do the weights predict the size of the barrier? (not supported)

The midpoint-erasure mechanism (Section 4.2) suggests that the barrier left by the best flip-free permutation should grow with $M = \sum_{i \in F} |s_i|/2$, the bound on the midpoint logit shift summed over the flipped units F.

Pre-registered test. Spearman $\rho(M, B_F)$, where B_F is the error barrier with the exact_sign permutation applied without its flips, pooled over tasks. Over 129 pairs, $\rho = 0.007$ (95% bootstrap interval $[-0.18, 0.19]$). The pre-registered rule labels this result "weak" because the interval includes zero. Since ρ is indistinguishable from zero, I report H7 as **not supported**, a more conservative wording than the rule requires. In the sensitivity runs, $\rho = -0.16$ $[-0.35, 0.03]$ at 1% and -0.21 $[-0.37, -0.05]$ at 5%, consistent with the ceiling effect below; the loss-based re-test was run at 2% only.

Why it failed (post hoc). B_F sits near each task's positive rate: median 17–22 points against positive rates of 18.8–22.1%. Once a substantial unit is erased at the midpoint, the blend predicts "negative" almost everywhere. The error barrier saturates at the trivial classifier and cannot grow with M. The ordering tests (H5) are immune to this ceiling, but magnitudes are not. The design choice to test H7 on the error barrier was mine and was a mistake.

Exploratory re-test (D8, decided after seeing the result). Repeating the test on loss barriers, which have no such ceiling, the mechanism does predict magnitude:

- raw loss: $\rho(M, B_F) = 0.86$ (95% interval $[0.79, 0.90]$, 129 pairs), positive in every task (0.70–0.94);
- calibrated loss: $\rho = 0.65$ ($[0.54, 0.75]$), also positive in every task (0.52–0.84).

Raw losses grow with the overall weight scale, and so does M, so part of the raw correlation may reflect scale. The calibrated loss removes most of the output-scale effect and still gives $\rho = 0.65$. Because this test was chosen after H7 failed, it supports the mechanism only as an exploratory result. A confirmatory test needs a fresh, pre-registered run.

6 Pitfalls

Training length inflates loss barriers. The coffee labels are separable, so cross-entropy keeps growing the weights: after 3,000 steps the median weight norm (input and output weights of included networks) rises from 32.2 at width 3 to 66.5 at width 64 (Appendix C). A misclassified point's loss grows with the weight scale; for linear predictors, Soudry et al. prove that gradient descent on separable data drives the weight norm to infinity, so that the loss on misclassified points keeps increasing (their Lemma 1 and Section 3) [27]. Raw loss barriers depend on training length and would mix that into the width trend. Zhao et al. make a related

point precise: with a rescaling symmetry, two parameter settings of the same function can have an arbitrarily large loss barrier between them [19, Proposition 5.3]. I therefore base every verdict on the error barrier and report loss barriers only after choosing the best of 41 output scales (0.25 to 8, log-spaced) at each blend point, an idea similar in spirit to the temperature calibration of Ito et al. [20]. This range proved too narrow for 43 of 131 unaligned pairs, whose best scale sat at the lower edge, so calibrated losses for unaligned pairs are only partly calibrated. Aligned pairs almost never hit the edge.

Weight decay. A side-check with L2 regularization (rule dated before the run) is reported in Appendix C; no network met the inclusion rule, so it gives no verdict and does not bear on the findings of Ito et al.

7 Robustness

- **Inclusion threshold (Study 1).** Rerunning the Study 1 analysis at 1% and 5% inclusion changes no verdict. (In Study 2, two H4a verdicts change; Section 5.1.) Two qualifications: at 1%, the width-3 paired intervals for the flip effect reach zero at their upper end (wm_sign − wm: −17.70 [−22.16, 0.00]), because 3 of 9 pairs needed no flip at all, although the count-based Q4 answer stands; at 5%, H1 holds with 10 of 12 pairs.
- **Test set.** The test-error barrier tracks the training-error barrier at every width (for example, wm at width 3: 17.95 training, 18.95 test).
- **Grid density.** In Study 2 (pre-registered), re-measuring every alignment at h^* on 97- and 385-point grids changes no verdict: the largest change in an exact_sign residual is 0.02 points, and calibration never hit the edge of its scale grid for an aligned pair (5 of 165 unaligned pairs did). In Study 1 (post-hoc, 15 Oct), re-measuring the width-3 exhaustive-search alignments on nested 97- and 385-point grids moves the maximum residual from 0.11 to 0.12 points (2% inclusion) and from 0.47 to 0.52 (5%). The 25-point barrier is a close lower bound.
- **Reproduction.** Retraining the original 40 coffee seeds in Study 2 reproduced Study 1's residuals exactly. A clean copy of the project, run from one script on the same machine, reproduced the main result files byte for byte (see the Reproducibility statement).

8 Limitations

- One architecture (one hidden layer, sigmoid), two-dimensional convex-polygon tasks with separable labels, and one optimizer setting (3,000 full-batch Adam steps). I did not test tanh (related by an affine change of parameters), depth or real data.
- The thin networks of Ainsworth et al. are ReLU networks, which have no sign flip; this study answers only the sigmoid analogue of their question.
- Minimal width is operational and depends on the error tolerance (Section 5); the claim fails for one task at each sensitivity threshold.
- "Exact symmetry" means equality of the function on all inputs. Approximate, data-dependent symmetries [14] are not searched. Above h^* the finite search does not cover the near-symmetries of nearly reducible networks, and no claim is made there.
- The claim covers networks that reach the inclusion threshold; the classification of the others cannot separate unfinished training from other minima.

- The coffee task in Study 2 includes Study 1's 11 pairs, so only the four polygon tasks are new tests. For the hexagon, $h^* = 4$ is an upper bound (the smallest width trained).
- The Study 2 model files are not released (they are rebuilt by `src/train_v2.py` and fingerprinted in the freeze file); the Study 2 code was tested on made-up data (`tests/test_v2.py`) and checked against the Study 1 search (D6).
- The size of the barrier is not predicted from the weights in error terms (H7). The loss-based result is exploratory.
- The Study 1 rules were recorded in a private notebook; only the Study 2 rules are independently timestamped. Decisions taken after seeing results are labelled and dated.

9 Conclusion

In minimal-width sigmoid networks, the barrier that permutation matching leaves is not a failure of the search but a symmetry missing from the matcher. Across five tasks and minimal widths 3 and 4, an exhaustive search over permutations and sign flips removed it in almost every pair whose networks learned the same lines (exploratory: 328 of 331). Where they learned clearly different lines, a residual of 2.38–9.58 points remained. The size of the barrier, measured as error, is not predicted by the weights, because the error barrier saturates. Measured as loss, it is predicted (exploratory, $\rho = 0.86$; calibrated 0.65).

Three questions follow. First, what does a teacher–student analysis in the style of Zhan et al. [17] predict for sigmoid or tanh networks, where sign flips must enter the matching? Second, does the picture survive depth and real data? Third, ReLU has no sign flip, but Theiss et al. show that a minimal-width ReLU network can have an oppositely oriented counterpart computing the same function that no permutation or rescaling reaches [13, Example B.13]. Ramakrishnan gives the complete classification of exact symmetries for one-hidden-layer ReLU networks: beyond permutations and positive rescalings, the only class comes from $\sigma(z) - \sigma(-z) = z$, and for a dense set of parameters none of it applies [28, Theorem 1]. Exact counterparts of this kind are therefore non-generic, but approximate ones may arise in training. Do units that learn the same line with opposite orientation leave a straight-line barrier in ReLU networks, as sign flips do here? The tools of this study (orientation-free line matching and a per-unit view of the midpoint) carry over directly to that question.

Acknowledgments

This study builds directly on the work of Entezari et al., Ainsworth et al. and Benzing et al., whose clearly stated open questions motivated it, and on the theoretical results of Sussmann, Şimşek et al., Zhan et al. and Zhao et al. I am grateful to all of these authors for making their work openly available. I thank Flavio Martinelli for pointing out that the flip step is the constant-plus-odd symmetry of Expand-and-Cluster, which his work already uses for alignment, and for pointing me to Theiss et al. The coffee-roasting task comes from the Machine Learning Specialization by Andrew Ng, DeepLearning.AI and Stanford Online.

Reproducibility statement

Code, result files and figures are released with this paper at `https://github.com/MrigeshLakhanpal/sign-flips-lmc` (archived on Zenodo, doi:10.5281/zenodo.23314245); the dated research notebook is available on request. A single script (`run.sh` on Linux and macOS, `run.ps1` on Windows) runs the unit tests, the learning-rate check, training, every pre-registered and post-hoc analysis except the optional

exhaustive search (`src/run_b1.py`, several hours), the bootstrap intervals, the figures, the inclusion sensitivity check, the spare-neuron figure and the weight-decay side-check. It took 87 minutes on a Windows laptop (CPU only, NumPy). Run from a clean copy of the project on the same machine, it produced byte-identical `barriers.csv` and `h1b.csv` and identical report tables. Software: Python 3.12.10, NumPy 2.5.3, SciPy 1.18.1, pandas 3.0.5, Matplotlib 3.11.2. All randomness is seeded. Study 2 runs in two phases, by design: `run_v2.ps1` (or `run_v2.sh`) runs the tests, the polygon construction, Stage 1 training and the freeze; after the freeze is committed, `run_v2_stage2.ps1` (or `.sh`) runs Stage 2 for every task and threshold, the verdicts, the exploratory loss re-test (`src/h7_explore.py`) and the dense-grid check, and `src/make_v2_tables.py` writes the Study 2 tables. The pre-registration, freeze, results and exploratory re-test are commits `349aa3a`, `4c07b33`, `b7b5669` and `a09afcf`. Model files are not released; they are fingerprinted in `results_v2/stage1_frozen.json` and rebuilt by `src/train_v2.py`. On the same Windows laptop, Stage 1 took about 40 minutes (estimated from commit times) and Stage 2 took 110 minutes; re-running Stage 2 from the frozen models rebuilt every file in `results_v2/` and both dense-grid outputs byte for byte. Training is deterministic on one machine; across platforms, BLAS differences can change low-order digits. A full reproduction of both studies therefore takes about four hours on a laptop CPU, plus several hours for the optional Study 1 exhaustive search.

Authorship and use of AI assistance

The research questions and hypotheses are mine; I chose the pre-registered rules, several from options Claude proposed, and ran every experiment on my own machine. I used Claude (Anthropic) as an AI research and coding assistant: it wrote most of the code and analysis scripts, which I tested and parts of which I wrote or re-implemented myself as checks; it searched for and summarized related work, of which I read the sections this paper relies on; it wrote the proof of Proposition 1, whose lemma I derived on paper with its guidance and tested on my own trained network (Figure 4); and it drafted the text from my research notes, which I reviewed line by line and approved. A separate Claude instance audited every number against the result files. For v2, Claude drafted the pre-registration, which I reviewed and committed; it wrote the v2 code, tested only on made-up data; I ran every v2 experiment; and Claude read the results from the public repository and drafted the v2 text, which I reviewed. Before release, five independent Claude reviewers audited the v2 code against the rules, recomputed every number from the result files, checked every citation and quotation against the cited papers, searched for prior work, and checked the text for unattributed overlap with sources; their findings were incorporated. A novelty audit produced by another Claude model (15 October) shaped the v2 plan; I accepted some of its points and rejected others, and the accepted ones became the v2 rules. No third-party code is included. I am responsible for the content.

References

[1] Rahim Entezari, Hanie Sedghi, Olga Saukh, and Behnam Neyshabur. The role of permutation invariance in linear mode connectivity of neural networks. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2110.06296.

[2] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M. Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 3259–3269, 2020.

[3] Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git Re-Basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2209.04836.

[4] Frederik Benzing, Simon Schug, Robert Meier, Johannes von Oswald, Yassir Akram, Nicolas Zucchet, Laurence

Aitchison, and Angelika Steger. Random initialisations performing above chance and how to find them. In *14th Annual Workshop on Optimization for Machine Learning (OPT 2022)*, 2022. arXiv:2209.07509.

[5] Odysseas Boufalis, Jorge Carrasco-Pollo, Joshua Rosenthal, Eduardo Terrés-Caballero, and Alejandro García-Castellanos. Symmetry-aware graph metanetwork autoencoders: Model merging through parameter canonicalization. In *Proceedings of the 1st Conference on Topology, Algebra, and Geometry in Data Science (TAG-DS)*, 2025. arXiv:2511.12601.

[6] Héctor J. Sussmann. Uniqueness of the weights for minimal feedforward nets with a given input-output map. *Neural Networks*, 5(4):589–593, 1992.

[7] Verner Vlačić and Helmut Bölcskei. Affine symmetries and neural network identifiability. arXiv:2006.11727, 2020.

[8] Fabrizio Pittorino, Antonio Ferraro, Gabriele Perugini, Christoph Feinauer, Carlo Baldassi, and Riccardo Zecchina. Deep networks on toroids: Removing symmetries reveals the structure of flat regions in the landscape geometry. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, 2022. arXiv:2202.03038.

[9] Ryuichi Kanoh and Mahito Sugiyama. Linear mode connectivity in differentiable tree ensembles. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2405.14596.

[10] Robert Hecht-Nielsen. On the algebraic structure of feedforward network weight spaces. In *Advanced Neural Computers*, pages 129–135. Elsevier, 1990.

[11] An Mei Chen, Haw-minn Lu, and Robert Hecht-Nielsen. On the geometry of feedforward neural network error surfaces. *Neural Computation*, 5(6):910–927, 1993.

[12] Flavio Martinelli, Berfin Şimşek, Wulfram Gerstner, and Johanni Brea. Expand-and-Cluster: Parameter recovery of neural networks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 34895–34919, 2024. arXiv:2304.12794.

[13] Marvin Theiss, Lukas Braun, Andrew M. Saxe, and Erin Grant. Parameter symmetries determine representational geometry in overparameterized nonlinear networks, 2026. arXiv:2609.39078.

[14] Vincent Bürgin, Daniel Herbst, Ya-Wei Eileen Lin, and Stefanie Jegelka. Beyond structural symmetries: Linear mode connectivity via neuron identifiability. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *PMLR*, 2026. arXiv:2606.04754.

[15] Jonas Gregor Wiese, Lisa Wimmer, Theodore Papamarkou, Bernd Bischl, Stephan Günnemann, and David Rügamer. Towards efficient MCMC sampling in Bayesian neural networks by exploiting symmetry. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, 2023. arXiv:2304.02902.

[16] Berfin Şimşek, François Ged, Arthur Jacot, Francesco Spadaro, Clément Hongler, Wulfram Gerstner, and Johanni Brea. Geometry of the loss landscape in overparameterized neural networks: Symmetries and invariances. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *PMLR*, pages 9722–9732, 2021. arXiv:2105.12221.

[17] Keyao Zhan, Puheng Li, and Lei Wu. Analyzing the role of permutation invariance in linear mode connectivity. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 258 of *PMLR*, 2025. arXiv:2503.06001.

[18] Zhanran Lin, Puheng Li, and Lei Wu. Exploring neural network landscapes: Star-shaped and geodesic connectivity. arXiv:2404.06391, 2024.

[19] Bo Zhao, Nima Dehmamy, Robin Walters, and Rose Yu. Understanding mode connectivity via parameter space symmetry. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025. arXiv:2505.23681.

[20] Akira Ito, Masanori Yamada, Daiki Chijiwa, and Atsutoshi Kumagai. Do we really need permutations? Impact of model width on linear mode connectivity. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.08023.

[21] Andrew Ng, DeepLearning.AI, and Stanford Online. Machine Learning Specialization, Course 2 (Advanced Learning Algorithms), Week 1 optional lab: coffee roasting. Coursera, 2022.

[22] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In

Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS), volume 9 of *PMLR*, pages 249–256, 2010.

[23] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[24] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2 (1–2):83–97, 1955.

[25] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[26] Charles Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904.

[27] Daniel Soudry, Elad Hoffer, Mor Shpigel Nacson, Suriya Gunasekar, and Nathan Srebro. The implicit bias of gradient descent on separable data. *Journal of Machine Learning Research*, 19:1–57, 2018.

[28] Pranavkrishnan Ramakrishnan. A complete symmetry classification of shallow ReLU networks, 2026. arXiv:2604.14037.

[29] Matthew Farrugia-Roberts. Functional equivalence and path connectivity of reducible hyperbolic tangent networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. arXiv:2305.05089.

A Full results

Table 8: Training-error barrier (points; median [95% bootstrap interval over pairs]), calibrated loss barrier (median) and test-error barrier (median, points) for every width and alignment mode. Generated directly from the result files.

Width	Mode	Pairs	Training-error barrier	Cal. loss barrier	Test-error barrier
3	bf_perm	11	17.88 [15.16, 20.22]	0.355	18.90
	bf_sign	11	0.00 [0.00, 0.05]	0.000	0.02
	none	11	20.05 [19.90, 21.50]	0.393	20.84
	wm	11	17.95 [16.07, 21.05]	0.283	18.95
	wm_sign	11	0.00 [0.00, 0.05]	0.000	0.02
4	bf_perm	20	7.77 [3.43, 15.39]	0.199	8.69
	bf_sign	20	0.08 [0.04, 0.10]	0.000	0.06
	none	20	20.13 [19.97, 20.44]	0.392	20.86
	wm	20	7.77 [3.43, 14.30]	0.227	8.69
	wm_sign	20	0.09 [0.04, 0.11]	0.000	0.06
6	bf_perm	20	4.68 [1.71, 6.15]	0.082	5.20
	bf_sign	20	0.39 [0.21, 0.48]	0.005	0.55
	none	20	20.23 [17.83, 20.33]	0.335	20.92
	wm	20	5.55 [1.89, 6.57]	0.106	5.98
	wm_sign	20	0.84 [0.34, 1.17]	0.012	1.08
8	bf_perm	20	1.94 [1.30, 2.98]	0.039	2.44
	none	20	20.09 [18.85, 20.24]	0.352	20.64
	wm	20	2.68 [1.46, 5.97]	0.070	3.55
	wm_sign	20	1.02 [0.54, 1.24]	0.017	1.05
16	none	20	14.38 [8.12, 19.45]	0.264	15.44
	wm	20	1.23 [0.91, 1.68]	0.025	1.45
	wm_sign	20	0.46 [0.31, 0.79]	0.008	0.57
32	none	20	11.10 [9.49, 13.08]	0.217	11.95
	wm	20	1.54 [0.83, 2.03]	0.033	1.61
	wm_sign	20	0.39 [0.26, 0.68]	0.007	0.28
64	none	20	9.29 [7.67, 15.60]	0.197	10.42
	wm	20	1.10 [0.86, 1.42]	0.023	1.20
	wm_sign	20	0.54 [0.31, 0.72]	0.010	0.45

Table 9: Weight matching minus brute force, per pair (points; median [95% interval]). Positive: weight matching is worse.

Width	Comparison	Pairs	Median per-pair difference
3	wm − bf_perm	11	0.00 [0.00, 0.00]
4	wm − bf_perm	20	0.00 [0.00, 0.00]
6	wm − bf_perm	20	0.01 [0.00, 0.77]
8	wm − bf_perm	20	0.63 [0.08, 1.34]
3	wm_sign − bf_sign	11	0.00 [0.00, 0.00]
4	wm_sign − bf_sign	20	0.00 [0.00, 0.00]
6	wm_sign − bf_sign	20	0.26 [0.00, 0.62]
3	wm − exact_perm	11	0.00 [0.00, 0.90]
4	wm − exact_perm	20	0.00 [0.00, 0.00]
6	wm − exact_perm	20	0.42 [0.05, 1.38]
8	wm − exact_perm	20	1.03 [0.42, 2.42]
3	wm_sign − exact_sign	11	0.00 [0.00, 0.00]
4	wm_sign − exact_sign	20	0.00 [0.00, 0.00]
6	wm_sign − exact_sign	20	0.34 [0.00, 0.66]

Table 10: With sign flips minus without, per pair (points; median [95% interval]). Negative: flips lower the barrier.

Width	Comparison	Pairs	Median per-pair difference
3	wm_sign − wm	11	−17.84 [−21.05, −16.07]
4	wm_sign − wm	20	−7.72 [−14.26, −3.30]
6	wm_sign − wm	20	−3.67 [−5.81, −0.91]
8	wm_sign − wm	20	−1.32 [−4.55, −0.54]
16	wm_sign − wm	20	−0.76 [−1.05, −0.36]
32	wm_sign − wm	20	−1.00 [−1.36, −0.47]
64	wm_sign − wm	20	−0.61 [−0.93, −0.38]
3	bf_sign − bf_perm	11	−17.79 [−20.18, −15.14]
4	bf_sign − bf_perm	20	−7.72 [−15.31, −3.35]
6	bf_sign − bf_perm	20	−3.37 [−5.80, −1.15]
3	exact_sign − exact_perm	11	−17.79 [−20.02, −15.14]
4	exact_sign − exact_perm	20	−7.72 [−12.26, −3.35]
6	exact_sign − exact_perm	20	−2.83 [−5.58, −0.93]

B One spare neuron connects all symmetry copies

In v2 this construction moved to an appendix: path existence follows from Farrugia-Roberts' Theorem 6.1, and the flip step is the constant-plus-odd symmetry of Martinelli et al., so its contribution is an explicit sigmoid construction and a segment count.

Two networks that differ by a sign flip compute the same function, yet, as shown above, the straight line between them has a large bump. This section shows that with one extra hidden unit there is a path between them along which the function never changes, and that combining this move with permutations reaches every symmetry copy. For single-hidden-layer tanh networks, the existence of such paths follows from a theorem of Farrugia-Roberts [29], and for sigmoid networks it follows from the same theorem through the linear map $\sigma(z) = \frac{1}{2}(1+\tanh(z/2))$; the contribution here is an explicit sigmoid construction with one spare unit and its segment count.

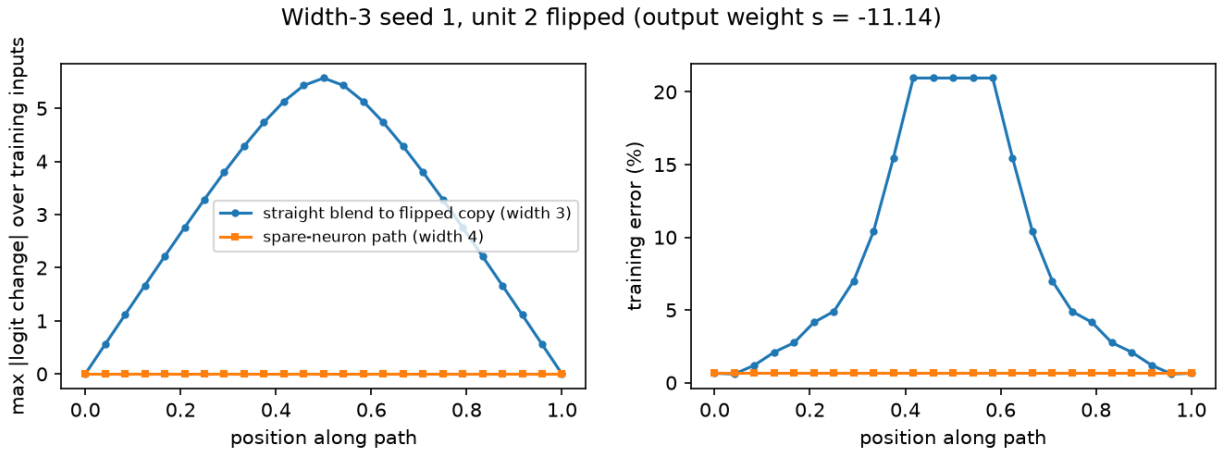


Figure 4: Width-3 network (seed 1), unit 2 flipped ($s = -11.14$). Left: largest change in the logit over the training inputs. Right: training error. Blue: straight blend to the flipped copy at width 3. Orange: the spare-neuron path of Lemma 1 at width 4, along which the function is unchanged.

Lemma 1 (Flip through a spare unit)**.** *Take unit i of Eq. (2) with parameters (w_i, b_i, s_i) and add a spare unit k with input parameters $(-w_i, -b_i)$ and output weight 0. For every t and every input, with $z = w_i^\top x + b_i$,*

$$(s_i - t)\,\sigma(z) \;-\; t\,\sigma(-z) \;+\; (c + t) \;=\; s_i\,\sigma(z) + c. \tag{4}$$

Proof. $-t\,\sigma(-z) + t = -t\,(1 - \sigma(z)) + t = t\,\sigma(z)$. □

Sliding t from 0 to s_i along this straight segment, $(s_i, s_k, c) = (s_i - t, -t, c + t)$, moves the whole contribution of unit i onto the spare: at $t = s_i$, unit i has output weight zero and the spare is exactly the sign-flipped unit i of Eq. (3). On the trained width-3 network of Section 4.2, this path changes the logit by about 5×10^{-15} (rounding error) and keeps the training error at 0.65% throughout, while the straight blend to the flipped copy peaks at 20.95% (Figure 4).

Proposition 1 (One spare neuron connects all permutation and sign-flip copies)**.** *Let f be a one-hidden-layer sigmoid network of width r with an output bias, embedded in width $r + 1$ by adding a unit with output weight 0. Then each of the $2^r r!$ permutation and sign-flip copies of f, embedded in the same way, is joined to f by a piecewise-linear path in parameter space along which the network function is exactly constant for every input. The path has at most $10r - 6$ segments if the spare unit's input parameters at the end may take any value (they do not affect the function), and at most $10r - 5$ if they must take a prescribed value, which one further segment sets.*

Proof. Three moves keep the function exactly constant along a straight segment:

(M1) A unit with output weight 0 may move its input weights and bias anywhere.

(M2) Two units with identical input weights and biases may trade output weight, keeping the sum fixed, since $(a + b)\,\sigma(z)$ depends only on $a + b$.

(M3) If unit k has input parameters $(-w_i, -b_i)$ and output weight 0, then Lemma 1 moves the contribution of unit i onto k, flipped.

Let k be the spare unit. A *transposition* of units i and j takes six segments: M1 (copy i's input parameters into k), M2 ($i \to k$), M1 (copy j's into i), M2 ($j \to i$), M1 (copy k's into j), M2 ($k \to j$); the spare again has output weight zero at the end. A *flip* of unit i takes four: M1 (set k to $(-w_i, -b_i)$), M3, M1 (copy k's input parameters

into i), M2 ($k \to i$). Any permutation of r elements is a product of at most $r-1$ transpositions, and any sign pattern is a product of at most r single-unit flips, so every copy is reached in at most $6(r-1)+4r = 10r-6$ segments. □

The permutation part follows the idea of Şimşek et al. [16], whose construction connects permutation copies through straight segments; the flip move M3 shifts output weight continuously between two units with opposite input parameters, which Martinelli et al. identify as a symmetry of constant-plus-odd activations such as the sigmoid [12, Appendix A.2.2] (a constant-duplicate-neuron group in the terminology of Theiss et al. [13, Definition B.12]), so Lemma 1 follows directly from their analysis. Theiss et al. also relate two oppositely oriented ReLU parameterizations of one function, by adjoining an auxiliary parameterization that computes the zero function rather than by a path [13, Example B.13]. Most directly, Farrugia-Roberts proved that the functional equivalence class of any reducible single-hidden-layer tanh network is piecewise-linear path-connected [29, Theorem 6.1]; a width-r network with a spare unit of output weight zero is reducible, so for tanh the existence part of Proposition 1 is his result. His construction uses a six-segment path per sign flip and a nine-segment path per transposition, and with at least half of the units blank the diameter is at most 7 segments (his Theorem 6.3); his Discussion notes that "Future work could investigate the trade-offs between shortest path length and rank [...]". Proposition 1 addresses the case with a single spare unit, for the sigmoid, with four segments per flip and six per transposition. The construction was also checked numerically on a random width-3 network with 5,000 random inputs, applying two transpositions and two flips: the largest change in the output along the whole path was 5.3×10^{-15}, and the end point equalled the target copy exactly. Two limits apply. The proposition concerns a network embedded with an inactive spare unit (output weight zero), not two independently trained networks of width $r+1$. It shows that a flat path *exists*; it does not address the straight-line barrier, which is what the experiments measure. In the language of Zhao et al. [19], for an irreducible network all $2^r r!$ copies are isolated points of the set of equivalent networks at width r, and one extra unit joins them with an explicit curve of constant function, in the spirit of their symmetry-induced curves (Section 6.1).

C Weight norms and the weight-decay side-check

Weight decay. Ito et al. [20] report that sufficiently wide image networks (MLPs, VGG-11 and ResNet-20 widened up to 16–32×) become linearly connected without permutations after temperature calibration, using weight decay 0.003 in most runs. With weaker decay (10^{-4}), VGG-11 and ResNet-20 show large barriers (their Section 5.4), although their Appendix F.9 notes that "for sufficiently wide models, smaller weight decay results in smaller barriers". Their explanation is stated for bias-free ReLU networks with low-rank weights. My main run used no weight decay. As a side-check with a rule dated before the run, I trained 40 width-64 networks with L2 regularization 0.003 on the weights (not the biases) added to the gradient inside Adam (not decoupled weight decay, so not a replication of any published setting). **None of the 40 networks met the inclusion rule**: all stopped at 6.15–6.80% training error, with weights about 7 times smaller (median norm 9.4 against 66.5). With no included pairs, the dated rule gives no verdict. An exploratory comparison on all 40 networks, ignoring inclusion and decided after seeing this result, shows no clear change in the unaligned barrier (median 9.29 without decay against 14.32 with it; interval of the difference $[-1.30, 6.68]$). These networks are underfit and roughly a hundred times narrower than those of Ito et al., so this side-check does not bear on their findings.

Table 11: Median weight norm (input and output weights, biases excluded) of included networks after 3,000 steps, no weight decay.

Width	3	4	6	8	16	32	64
Median norm	32.19	33.69	38.56	41.02	46.88	54.96	66.50

Table 12: Exploratory weight-decay comparison at width 64 (all 40 networks, inclusion ignored; 20 pairs with the same initializations as the main run). Medians; interval for median(decay) − median(base). Decided after seeing that no decay network met the inclusion rule.

Mode	Measure	No decay	L2 0.003	95% interval
none	error barrier (points)	9.29	14.32	[−1.30, 6.68]
none	calibrated loss barrier	0.197	0.244	[−0.024, 0.074]
wm	error barrier (points)	1.10	1.31	[−0.67, 2.20]
wm	calibrated loss barrier	0.023	0.020	[−0.013, 0.016]
wm_sign	error barrier (points)	0.54	0.12	[−0.68, 0.32]
wm_sign	calibrated loss barrier	0.010	0.002	[−0.015, 0.001]

D Study 2: further pre-registered outputs

Table 13: Study 2 Stage 1: networks meeting each inclusion threshold, by task and width (training error only; `results_v2/stage1.csv`).

Task	Width	Trained	$\leq 1\%$	$\leq 2\%$	$\leq 5\%$
Coffee	3	120	56	72	77
Triangle	2	100	0	0	0
Triangle	3	100	93	94	94
Triangle	4	100	100	100	100
Triangle	5	100	100	100	100
Quadrilateral	2	100	0	0	0
Quadrilateral	3	100	0	0	97
Quadrilateral	4	100	56	65	100
Quadrilateral	5	100	57	83	100
Quadrilateral	6	100	83	95	100
Pentagon	3	100	0	0	93
Pentagon	4	100	0	43	99
Pentagon	5	100	36	79	100
Pentagon	6	100	48	93	100
Pentagon	7	100	66	97	100
Hexagon	4	100	0	59	100
Hexagon	5	100	5	85	100
Hexagon	6	100	61	93	100
Hexagon	7	100	77	95	100
Hexagon	8	100	89	99	100

Flip counts. At h^* (2%), the exact_sign alignment flipped 0, 1, 2 and 3 units in 36, 80, 48 and 1 of the 165 pairs.

Widths above h^* (descriptive, no claim). Median training-error barrier, wm / wm_sign, in points: triangle 3.37 / 0.68 (width 4) and 2.72 / 1.15 (5); quadrilateral 6.22 / 1.04 (5) and 8.23 / 1.23 (6); pentagon 4.47 / 0.10 (5), 5.11 / 0.27 (6) and 5.73 / 0.77 (7); hexagon 4.24 / 0.68 (5), 4.83 / 0.09 (6), 3.78 / 0.47 (7) and 3.76 / 0.52 (8). Above the minimal width, sign-aware matching still lowers the barrier, but it is not always removed, as in Study 1 at widths 6–64; the finite search does not cover the near-symmetries of these wider networks.

E Timeline of decisions

Table 14: When each decision was taken. Post-hoc decisions are marked.

Date (2026)	Decision
4–5 Oct	Hypotheses H1–H3 and questions Q1–Q4 fixed and recorded, before any training
7 Oct	Learning-rate rule written, then checked (10 of 10 runs passed); coffee rule compared with an unofficial public copy of the lab file
8 Oct	Inclusion rule (2%), line tolerances (10°, 0.25), $\varepsilon = 0.5$, H2 band (0.5–3), H1b distance and test; training-error barrier chosen as the deciding quantity
9 Oct	Main run; pre-registered verdicts written; inclusion sensitivity (1% and 5%); rules for the complete-group search dated before it ran
9–10 Oct	Complete-group search; decision not to run the signed width-8 search (post-hoc)
10 Oct	Weight-decay rule dated
11 Oct	Weight-decay run; exploratory comparison decided after the result (post-hoc); bootstrap intervals; independent numeric audit of the draft; reproduction from a clean copy; proof of Proposition 1
12 Oct	Comparison of flip counts with a fair-coin model (post-hoc); independent audits of numbers, code and references
13 Oct	Credited the flip step to Martinelli et al. and cited Theiss et al., after correspondence with F. Martinelli
14 Oct	Citation audit: credited the existence of the spare-unit paths (for tanh) to Farrugia-Roberts; cited Kanoh and Sugiyama, Wiese et al. and Soudry et al.
15 Oct	Wording after an external review: H3 stated as an endpoint result, sigmoid path existence inherited from tanh, segment count with prescribed spare endpoint ($10r - 5$), operational definition of minimal width, 25-point grid stated, inclusion counts in abstract
15 Oct	Novelty audit received; v2 plan decided (title rule, polygon tasks, measures, excluded-run classes)
19 Oct	v2 rules committed (`349aa3a`) before any v2 training; Stage 1 frozen (`4c07b33`); D7 (sensitivity h^*) written after the training counts, before any barrier; Stage 2 run (`b7b5669`); D8 (H7 on loss) and D9 (same-line split) decided after the results (post-hoc)